

1.3.5 Print Versus Return

Show Lecture Video

A common mistake is to confuse `print` with `return`. For example:

```
1 def cubed(x):  
2     print(x**3) # Here is the error!  
3  
4 cubed(2) # prints 8, so it seems to work....
```

Run

This code *seems* to work, but it doesn't! The function does not explicitly return a value. Thus, as we have learned, Python automatically returns `None`. Let's see that:

```
1 def cubed(x):  
2     print(x**3) # Here is the error!  
3  
4 print(cubed(2)) # still prints 8, but then also p
```

Run

As the comment notes, this code still prints `8` but it also prints `None`. That suggests something went wrong. Let's go one step further and really prove it:

```
1 def cubed(x):  
2     print(x**3) # Here is the error!  
3  
4 x = cubed(2)  
5 print(x + 1) # crashes: we cannot add None and an
```

Run

Ok, so this proves that the function returned `None`. The key point is: **calling `print` has no effect on a return value.**

The fix is simple: do not use `print` to return a value. Instead, always use `return`, like so:

```
1 def cubed(x):
```

```
1 def cubed(x):  
2     return x**3 # Fixed  
3  
4 x = cubed(2)  
5 print(x + 1) # print 9 (whew!)
```

Run

Question: How many variables are in the previous example?

Show answer

Checkpoint 1

What will the following code print?

```
def f(x):  
    x += 10  
  
print(f(10))
```

- ☐ 20
- ☐ This will crash because the function `f` doesn't print or return.
- ☐ None
- ☐ 10

Submit

Continue